Feuille de TP n° 1

Bases de la programmation – Prise en main

Mise en route

1. Lancez un terminal pour pouvoir exécuter des commandes Linux.
2. Vous êtes à présent dans votre répertoire personnel (« home »).
3. Créez un répertoire (ou dossier) appelé `BDP_Python` ; pour ce faire, tapez : `mkdir BDP_Python`
4. Allez dans le répertoire `BDP_Python` ; pour ce faire, tapez : `cd BDP_Python`
5. Vous êtes maintenant dans le répertoire `BDP_Python`. Créez-y un répertoire que vous appellerez `TP1` (en utilisant la commande `mkdir`).
6. Allez dans le répertoire `TP1` (en utilisant la commande `cd`).
7. Lancez un éditeur de texte (`code`, `gedit` ou `geany`) en tapant son nom suivi du nom du fichier, par exemple `gedit exo1.py &` (le `&` rend la main au terminal).
8. Vous pouvez commencer à programmer.
9. Enregistrez votre programme dans le fichier `exo1.py` (un fichier par exercice : `exo2.py`, `exo3.py`, ...).
10. Exécutez-le en tapant dans le terminal : `python3 exo1.py`

Programmez sans regarder la correction. Vous pourrez comparer votre programme avec la correction en cas de problème que vous n'arrivez pas à déceler.

Rappels Python

Afficher. L'instruction `print` affiche ses arguments, séparés par des espaces :

```
A = 12
print("Bonjour")
print(A)
print("La valeur contenue dans la variable A est", A)
```

Ce qui s'affiche à l'écran :

```
Bonjour
12
La valeur contenue dans la variable A est 12
```

Lire une valeur au clavier. L'instruction `input` affiche un message, attend que l'utilisateur tape une valeur, puis renvoie cette valeur sous la forme d'une *chaîne de caractères*. Pour obtenir un nombre, il faut la convertir avec `int` (entier) ou `float` (réel) :

```
a = input("Entrez une valeur : ")        # a est une chaîne de caractères
n = int(input("Entrez un entier : "))    # n est un entier
x = float(input("Entrez un réel : "))    # x est un réel
```

Racine carrée. La fonction `sqrt` du module `math` calcule une racine carrée :

```
from math import sqrt
print(sqrt(16))                    # affiche 4.0
```

Exercice 1 Comparaison de deux entiers

Écrire un programme qui demande à l'utilisateur deux entiers, stockés dans deux variables `a` et `b`, et qui affiche « plus grand » si `a` est strictement supérieur à `b`. Dans le cas contraire, le programme affiche « plus petit ou égal ».

Modifier ensuite le programme afin qu'il affiche l'un des trois messages suivants : « plus grand », « plus petit » ou « égal ».

Exercice 2 Aire d'un rectangle

Écrire un programme qui demande à l'utilisateur une longueur et une largeur (deux nombres réels), stockées dans deux variables `longueur` et `largeur`. La longueur doit être supérieure ou égale à la largeur.

Si cette condition est respectée, le programme affiche l'aire du rectangle correspondant. Sinon, il affiche le message « Données incohérentes ».

Exercice 3 Contrôle de saisie

Écrire un programme qui lit six entiers et les stocke dans six variables : `a`, `b`, `c`, `d`, `e` et `f`.

Si au moins trois de ces variables sont non nulles, le programme affiche « La saisie est correcte ». Sinon, il affiche « La saisie est incorrecte ».

Exercice 4 Tri de trois nombres

Écrire un programme qui lit trois nombres et les stocke dans trois variables `a`, `b` et `c`. Le programme doit ensuite réordonner leur contenu de manière à obtenir :

$$a \leq b \leq c.$$

Afficher les trois variables à la fin du programme afin de vérifier le résultat.

Exercice 5 Minimum, maximum et moyenne

Écrire un programme qui permet de saisir dix valeurs réelles et qui affiche la plus petite valeur, la plus grande valeur et la moyenne des dix valeurs. Les boucles n'ayant pas encore été vues, on répétera dix fois le bloc de saisie. Le programme ne doit utiliser que **quatre variables** : la valeur saisie, le minimum, le maximum et la somme.

Exercice 6 Distance entre deux points

Écrire un programme qui saisit les coordonnées entières de deux points du plan et qui affiche la distance les séparant sous la forme d'un nombre réel (voir les rappels pour la racine carrée).

Exercice 7 Triangle rectangle

Écrire un programme qui saisit les coordonnées entières de trois points du plan et qui indique si ces trois points forment un triangle rectangle. On utilisera le théorème de Pythagore sur les carrés des longueurs des côtés (aucune racine carrée n'est nécessaire). Le programme affiche « Triangle rectangle » ou « Pas un triangle rectangle » ; si deux des points sont identiques, il affiche « Deux points sont confondus ».

Exercice 8 Calculatrice

Écrire un programme qui demande à l'utilisateur une opération parmi `+`, `-`, `*` et `/`, puis deux opérandes entiers. Le programme affiche le résultat de l'opération appliquée aux deux opérandes ; pour la division, il affiche le quotient entier (`//`). Si l'opération n'est pas l'une des quatre, le programme affiche « Opération inconnue ».

> **Attention :** pour la division, pensez à interdire la division par zéro.

Exemple d'exécution :

```
Entrez l'opération : *
Entrez le premier opérande : 3
Entrez le deuxième opérande : 4
Le résultat de 3 * 4 est 12
```

Exercice 9 Équation du second degré

Écrire un programme qui demande trois valeurs réelles `a`, `b` et `c`, puis affiche les solutions réelles de l'équation

$$ax^2 + bx + c = 0.$$

Si `a` est nul, le programme affiche « a doit être différent de zéro » et s'arrête. Sinon, selon le signe du discriminant $\Delta = b^2 - 4ac$, il affiche « Pas de solution réelle » ($\Delta < 0$), la solution double ($\Delta = 0$) ou les deux solutions ($\Delta > 0$).

Exercice 10 Chiffres d'un nombre

Écrire un programme qui reçoit un entier positif à quatre chiffres et qui affiche ses quatre chiffres, séparés par des espaces.

Le programme doit d'abord vérifier que le nombre contient bien quatre chiffres. Dans le cas contraire, il affiche : « Le nombre ne contient pas 4 chiffres ».

Exemple : pour 1234, le programme affiche `1 2 3 4`.

Feuille de TP n° 2

Bases de la programmation – Les fonctions

Rappels Python

Définir et appeler une fonction. Une fonction se définit avec `def`, ses paramètres entre parenthèses et un bloc indenté ; `return` renvoie une valeur à l'appelant et termine immédiatement la fonction :

```
def aire_rectangle(largeur, hauteur):    # définition
    aire = largeur * hauteur
    return aire                          # valeur renvoyée

a = aire_rectangle(3, 4)                 # appel : a vaut 12
print(aire_rectangle(2, 5))              # affiche 10
```

Renvoyer n'est pas afficher. `print` affiche à l'écran mais ne renvoie rien ; une fonction qui doit « donner » un résultat utilise `return`. Une fonction peut renvoyer un booléen ou plusieurs valeurs à la fois :

```
def est_pair(n):
    return n % 2 == 0                    # renvoie True ou False

def division(a, b):
    return a // b, a % b                 # renvoie deux valeurs

q, r = division(17, 5)                   # q vaut 3, r vaut 2
```

Exercice 1 Équation du second degré

On souhaite résoudre l'équation $ax^2 + bx + c = 0$ pour tous réels `a`, `b` et `c`, y compris lorsque `a` ou `b` sont nuls. La méthode est la suivante (S désigne l'ensemble des solutions) :

Entrée : trois réels a, b et c
Sortie : l'ensemble S des solutions de $ax^2 + bx + c = 0$

si $a = 0$ **alors**
  si $b = 0$ **alors**
    si $c = 0$ **alors** $S \leftarrow \mathbb{R}$ (tous les réels sont solutions)
    sinon $S \leftarrow \emptyset$ (pas de solution)
  sinon
    $S \leftarrow \{ -c/b \}$
sinon
  $\Delta \leftarrow b^2 - 4ac$
  si $\Delta = 0$ **alors** $S \leftarrow \{ -b/(2a) \}$
  sinon si $\Delta > 0$ **alors** $S \leftarrow \{ \ldots, \ldots \}$ (à compléter)
  sinon $S \leftarrow \emptyset$ (pas de solution réelle)

A. Écrire la fonction `discriminant(a, b, c)` qui renvoie le discriminant $\Delta = b^2 - 4ac$.

B. Écrire la fonction `resoudre(a, b, c)` qui résout l'équation. **Attention :** cette fonction ne doit

faire aucun affichage. Elle renvoie trois valeurs `nb, x1, x2` : `nb` est le nombre de solutions (-1 si tous les réels sont solutions, sinon 0, 1 ou 2) et `x1`, `x2` sont les solutions éventuelles (`x1 < x2` s'il y en a deux ; une valeur inutilisée vaut `None`).

C. Écrire une fonction d'affichage, sans paramètre, qui demande `a`, `b` et `c` à l'utilisateur, appelle `resoudre` puis affiche le résultat de la résolution. Exemple d'exécution :

```
Entrez a : 0
Entrez b : 2
Entrez c : 1
Votre équation admet une seule solution : -0.5
```

Exercice 2 Décomposition en facteurs premiers

Voici la méthode de décomposition d'un entier en facteurs premiers.

Entrée : un entier naturel $n > 1$
Sortie : une chaîne de caractères qui représente la décomposition de n en facteurs premiers

`resultat` ← chaîne vide
tant que $n > 1$ **faire**
  d ← le plus petit diviseur premier de n
  si `resultat` est vide **alors**
    mettre d dans `resultat`
  sinon
    ajouter `" x "` puis d à la fin de `resultat`
  $n \leftarrow n / d$ (division entière)
retourner `resultat`

A. Écrire une fonction `est_premier(n)` qui renvoie `True` si `n` est un nombre premier, `False` sinon.

B. Écrire une fonction `plus_petit_diviseur(n)` qui utilise `est_premier` et renvoie le plus petit diviseur premier de `n` (strictement supérieur à 1).

C. Écrire une fonction `decomposition_facteurs(n)` qui produit la décomposition en facteurs premiers de `n` en appelant plusieurs fois la fonction de la question B. Par exemple, pour $n = 12$, la fonction renvoie la chaîne `"2 x 2 x 3"`.

> **Indication :** ici, exceptionnellement, on utilise une variable de type `str` (chaîne de caractères). Au début, on écrit `resultat = ""`. Si `resultat` vaut à un moment `"2 x 2"` et que l'on veut ajouter l'entier `d` comme facteur, on écrit simplement `resultat = resultat + " x " + str(d)`.

Exercice 3 Le nombre mystère

On programme un jeu dans lequel l'utilisateur doit deviner, en un nombre limité de tentatives, un nombre entier tiré au hasard par l'ordinateur.

A. Écrire une fonction `generer_nombre(minimum, maximum)` qui renvoie un entier aléatoire compris entre `minimum` et `maximum` (inclus). On utilisera `random.randint(minimum, maximum)`.

B. Écrire une fonction `verifier_nombre(secret, proposition)` qui renvoie 1 si la proposition est plus petite que le nombre secret, 2 si elle est plus grande et 3 si elle est identique.

C. Écrire une fonction `jouer()` qui demande à l'utilisateur combien de tentatives il souhaite, puis appelle `generer_nombre` pour créer le nombre mystère (entre 1 et 100). Ensuite, elle utilise une

boucle pour que l'utilisateur propose un nombre à chaque tentative ; elle utilise `verifier_nombre` pour lui donner un indice à chaque essai (« Le nombre mystère est plus grand » ou « Le nombre mystère est plus petit »). Le jeu se termine si l'utilisateur trouve le nombre ou si les tentatives sont épuisées ; la fonction affiche alors un message final avec le résultat (gagné, en indiquant le nombre de tentatives utilisées, ou perdu, en révélant le nombre mystère).

D. Écrire la fonction `main()` qui lance une partie puis demande, après chaque partie, à l'utilisateur s'il veut continuer à jouer. La fonction `main` sera appelée dans le programme comme ceci :

```
# Lancer le programme
if __name__ == "__main__":
    main()
```

Indication : la condition `__name__ == "__main__"` est vraie lorsque le fichier est exécuté directement (`python3 exo3.py`). Elle permet de placer l'appel initial à la fin du fichier, après toutes les définitions de fonctions.

Exercice 4 Factorielle, coefficient binomial et triangle de Pascal

A. Écrire une fonction `factorielle` qui prend en argument un entier naturel n et qui renvoie $n!$. Rappel : $n! = 1 \times 2 \times \cdots \times n$; on ajoute $0! = 1$.

B. Écrire une fonction `coeff_binomial` qui prend en argument deux entiers naturels n et k (avec $k \leq n$) et qui renvoie le coefficient binomial correspondant, « k parmi n », donné par la formule

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!}.$$

Le résultat renvoyé doit être un entier.

C. Écrire une fonction `triangle_pascal` qui prend en argument un entier `nb_lignes` et qui affiche le triangle de Pascal avec `nb_lignes` lignes, les nombres d'une même ligne étant séparés par un espace. Voici un exemple avec 6 lignes :

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

En numérotant les lignes et les colonnes à partir de 0, le nombre situé sur la ligne numéro n et la colonne numéro k est le coefficient binomial $\binom{n}{k}$.

Une des applications du triangle de Pascal : les coefficients de la ligne numéro 4, soit 1 4 6 4 1, sont aussi les coefficients du développement

$$(a+b)^4 = a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4.$$

Feuille de TP n° 3

Bases de la programmation – Les listes : le Mastermind

Rappels Python

Des listes utilisées comme des tableaux. Dans ce cours d'introduction à la programmation, les listes servent à simuler le fonctionnement des tableaux de base tels qu'on les retrouve dans d'autres langages. Par conséquent, on évitera d'exploiter les fonctionnalités avancées des listes : seules les opérations `len`, `append`, l'accès par indice `[]` et `insert` sont autorisées (ni opérateur `in`, ni `remove`, `count`, `index`, tranches…). Ainsi, pour supprimer un élément, il faudra décaler manuellement vers la gauche tous les éléments situés à sa droite.

Lire une liste de chiffres tapés sur une ligne. `split()` découpe une chaîne aux espaces ; il reste à convertir chaque morceau en entier :

```
ligne = input("Entrez une liste de chiffres : ")   # par exemple : 1 2 3 4 5
mots = ligne.split()                 # ["1", "2", "3", "4", "5"]
H = []
for i in range(len(mots)):
    H.append(int(mots[i]))           # H vaut [1, 2, 3, 4, 5]
```

Renvoyer deux valeurs. `return bien, mal` dans une fonction, puis `bien, mal = Score(H, R)` à l'appel.

Dans ce TP, on vous propose de programmer le jeu du Mastermind.

Exercice 1 L'ordinateur fait deviner l'utilisateur

Principe : l'ordinateur crée aléatoirement une liste de 5 chiffres *différents*, et l'utilisateur doit deviner les 5 chiffres dans l'ordre. Pour cela, à chaque fois que l'utilisateur propose une liste, le programme lui dit combien de chiffres sont bien placés, et combien sont justes mais mal placés.

Exemple, en admettant que l'ordinateur a généré R = [1, 2, 4, 8, 5] :

```
Entrez une liste de chiffres : 1 2 3 4 5
3 bien placés, 1 mal placé
Entrez une liste de chiffres : 1 2 3 6 4
2 bien placés, 1 mal placé
Entrez une liste de chiffres : 1 2 4 8 5
5 bien placés, 0 mal placé
BRAVO !!
Une autre partie (o/n) ?
```

A. Écrire la fonction `Init(n, k)` qui renvoie une liste de `n` chiffres *différents* générée aléatoirement : la liste à deviner. Les nombres sont tirés dans l'intervalle $[1, k]$ (on suppose $k \geq n$). On pourra écrire d'abord une fonction `contient(L, v)` qui renvoie `True` si la valeur `v` figure dans la liste `L`, l'opérateur `in` étant interdit.

B. Écrire une fonction `saisir_proposition(n)` qui saisit la liste des `n` chiffres proposés par l'utilisateur, tapés sur une seule ligne et séparés par des espaces, et renvoie cette proposition sous la forme d'une liste Python `H`. Tant que la ligne ne contient pas exactement `n` valeurs, la saisie est redemandée.

C. Écrire une fonction `Score(H, R)` qui renvoie deux valeurs : le nombre de chiffres bien placés (indices `i` tels que `H[i] == R[i]`) et le nombre de chiffres présents mais mal placés (indices `i` tels que `H[i] != R[i]` et `H[i]` figure dans `R`). `H` est la proposition et `R` la liste de référence ; les deux listes ont la même taille et sont formées de chiffres différents.

D. Écrire une fonction `jeu(n, k)` qui réalise une partie (ici $n = 5$ et $k = 9$) et une fonction `main()` qui enchaîne les parties tant que l'utilisateur répond « o » à la question « Une autre partie ? ».

Exercice 2 L'utilisateur fait deviner l'ordinateur

Ici, on se propose d'implémenter une version dans laquelle l'utilisateur choisit une liste de chiffres différents que l'ordinateur doit deviner. Soit `R` la liste choisie par l'utilisateur, `Ens_H` l'ensemble des listes proposées par l'ordinateur jusqu'à l'instant t, et `Scores_H` l'ensemble des scores correspondants, donnés par l'utilisateur.

Une liste `H` reste *candidate* à l'instant $t + 1$ si elle est compatible avec toutes les réponses déjà obtenues : pour chaque proposition H_i de `Ens_H`, le score qu'aurait obtenu H_i si `H` était le code secret doit être égal au score que l'utilisateur a réellement donné pour H_i, soit

$$\forall H_i \in \text{Ens_H} : \text{Score}(H_i, H) = \text{Score}(H_i, R).$$

Fonctionnement global :

- l'utilisateur fixe une liste secrète `R` (sans la communiquer à l'ordinateur) ;
- l'ordinateur génère tous les candidats possibles : une liste de listes (toutes les possibilités `H` formées de chiffres différents) ;
- chaque fois que l'ordinateur propose une hypothèse `H`, l'utilisateur détermine à la main le score de `H` par rapport à `R` (nombre de bien placés, nombre de mal placés, même définition que `Score`) et le donne à l'ordinateur ;
- l'ordinateur stocke cette proposition dans `Ens_H` et son score dans `Scores_H` ;
- le coup suivant est choisi parmi les candidats restants, c'est-à-dire ceux qui respectent la règle ci-dessus : à chaque essai, l'ensemble des possibilités est réduit aux listes compatibles avec le score obtenu.

Pour commencer, on vous donne la fonction qui calcule toutes les listes de longueur `longueur` dont les chiffres sont choisis entre 1 et `n` (répétitions comprises). Tester cette fonction pour quelques valeurs de `n` et de `longueur`.

```
def toutes_possibilites(n, longueur):
    possibilites = [[]]
    for _ in range(longueur):
        nouvelles = []
        for c in range(len(possibilites)):
            comb = possibilites[c]
            for i in range(1, n + 1):
                L = list(comb)          # copie de comb
                L.append(i)
                nouvelles.append(L)
        possibilites = nouvelles
    return possibilites

# Exemple d'utilisation
print(toutes_possibilites(3, 3))
```

A. Écrire une fonction `chiffres_differents(L)` qui renvoie `True` si tous les éléments de `L` sont différents, puis une fonction `candidats(n, longueur)` qui renvoie la liste des possibilités formées de chiffres différents (les résultats de `toutes_possibilites` qui vérifient `chiffres_differents`).

B. Écrire la fonction `devine(longueur, n)` qui prend en argument la longueur du code et le nombre `n` (les chiffres possibles sont 1, …, `n`) et qui devine pas à pas le code fixé par l'utilisateur, en suivant l'algorithme ci-dessus : à chaque essai, elle propose la première liste encore candidate, demande à l'utilisateur le nombre de bien placés puis le nombre de mal placés, et réduit la liste des candidats. Elle renvoie deux valeurs : la liste que l'utilisateur avait fixée et le nombre d'essais qui ont été nécessaires pour la découvrir.

C. Écrire une fonction `main()` qui lance le jeu : elle demande à l'utilisateur de fixer un code secret (sans le dire à l'ordinateur, bien sûr) en donnant la longueur et la valeur de `n`, puis lance la fonction `devine` et affiche le résultat.

> **Attention :** la fonction fournie `toutes_possibilites` reçoit `n` *avant* la longueur, alors que `devine` reçoit la longueur avant `n`.